Reinforcement Learning

From Markov Decision Processes to Deep RL - Lecture 1/4

Etienne Chassaing

CentraleSupélec

How much do you know about Reinforcement Learning (RL)?

Plan

1. Introduction
2. Sequential Decision Making and RL framework
3. Markov Decision Processes (MDPs)
4. RL framework
5. Vocabulary
6. Bellman equations

Plan

1. Introduction
2. Sequential Decision Making and RL framework
3. Markov Decision Processes (MDPs)
4. RL framework
5. Vocabulary
6. Bellman equations

What does machine learning learn?

Machine learning

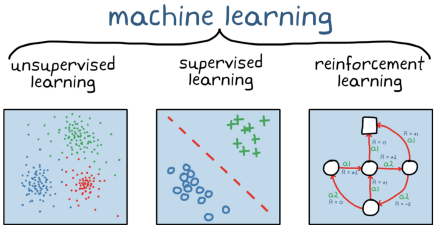
Machine learning is always the learning of a function (explicit or implicit, parametrized or not...): $y = f(x)$ where x and y are the input and output vectors, $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$.

What does machine learning learn?

Machine learning

Machine learning is always the learning of a function (explicit or implicit, parametrized or not...): $y = f(x)$ where x and y are the input and output vectors, $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$.

- **Unsupervised case**: learn f from inputs only, $x_1, \ldots, x_n$: Clustering, dimensionality reduction ...
- **Supervised case**: learn f from examples of x and y: $x_1, \ldots, x_n$ and labels $y_1, \ldots, y_n$: Classification, regression ...
- **Reinforcement learning**: learn how to act from environment observations $action = \pi(observations)$, learning is guided by a reward function

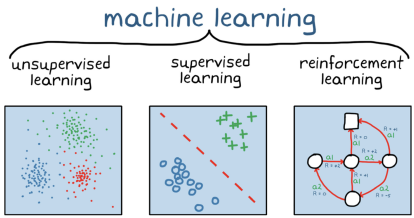


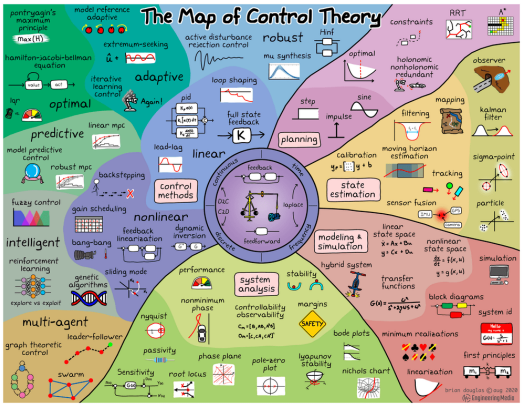
Supervised vs unsupervised vs reinforcement learning, Matlab RL book.

$\Longrightarrow$ Let's try to understand why RL does not fit either of the first two definitions

RL is both a subset of Machine Learning and of Control theory

Your class is about data-driven control, we'll oscillate between the two worlds throughout this course.





Source: MathWorks and Brian Douglas.

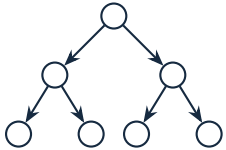
Sequential decision making

Definition

Sequential decision making describes a situation where the decision maker (DM) makes successive decisions based on the observation (complete or not) of the impact of previous decisions.

Examples that fit perfectly:

- ⇒ Game playing (Atari, Go, chess, shogi…)
- ⇒ Discrete decision making (robot navigation, path planning, maze solving…)
- ⇒ Continuous decision making (stock trading, action selection…)
- ⇒ Control problems (robot locomotion, driving, HVAC for buildings…)



A decision tree: each stage branches into the possible states, from different actions

RL is one way, among others, to navigate this tree and find the best path to the goal.

Illustrative example 1/2 : Atari games

Comments :

- RL **agent** interacts with the game with no prior information
- It can **act** on the game (right / left / do nothing)
- It can **perceive** the screen (pixels) and the score (reward)
- It learns to play the game by **trials and errors**, guided by the **score**



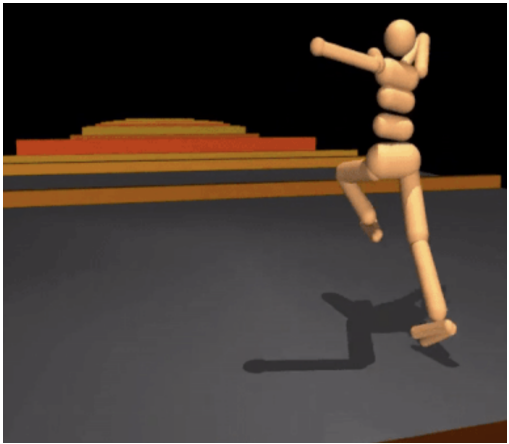
Atari Breakout gameplay (link)

Source: V. Mnih et al., “Playing Atari with Deep Reinforcement Learning”, arXiv:1312.5602 (link)

Illustrative example 2/2 : Locomotion

Comments:

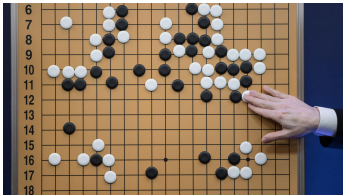
- Agent actions are now **wider** and continuous: each joint position
- Goal is to **move forward** and not to fall
- The obtained strategy looks **suboptimal** but does work
- The control strategy **jitters a lot**



Simulated humanoid locomotion (link)

Source: DeepMind, “Producing flexible behaviours in simulated environments” (link)

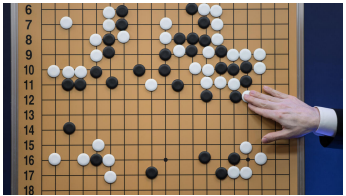
RL recently



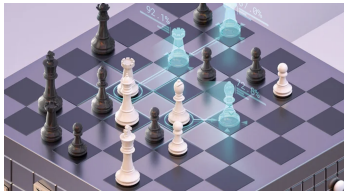
AlphaGo (2016) - David Silver et al.:

beats Lee Sedol at Go, **deep RL + Monte-Carlo Tree Search**

RL recently

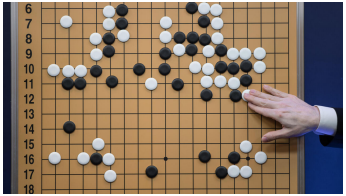


AlphaGo (2016) - David Silver et al.: beats Lee Sedol at Go, **deep RL + Monte-Carlo Tree Search**



AlphaZero - same architecture, trained purely by **self-play** (no human data), generalized to chess and shogi. **Too many moves to explore** for brute-force search. Did create a new chess opening, never sawn before in training dataset.

RL recently



AlphaGo (2016) - David Silver et al.: beats Lee Sedol at Go, **deep RL + Monte-Carlo Tree Search**



AlphaZero - same architecture, trained purely by **self-play** (no human data), generalized to chess and shogi. **Too many moves to explore** for brute-force search. Did create a new chess opening, never sawn before in training dataset.



Turing Award 2024 - awarded to R. Sutton and A. Barto for developing the conceptual and algorithmic foundations of **reinforcement learning** (and authored the bible of RL: Sutton & Barto, 1998/2018 on which this class is based: ☺)

Source: ACM A.M. Turing Award, 2024 (link)

Demis Hassabis and the 2024 Nobel Prize in Chemistry



Source: The Nobel Prize in Chemistry 2024 (link)

- Demis Hassabis (DeepMind co-founder, AlphaGo/AlphaZero) won the **2024 Nobel Prize in Chemistry**, jointly with John Jumper and David Baker
- Awarded for **AlphaFold**, which predicts **protein 3D structures** from their amino-acid sequence
- Hassabis is a **computer scientist**, not a chemist
- Computer Science and AI reshape other fields, as physics used to in the last century

RL history

A few ideas and dates to understand RL's development, along three intertwined threads:

- **Optimal decision making**: von Neumann & Morgenstern 1944 (game theory / expected utility); Wald 1950 (statistical decision theory); Nash 1950 (equilibrium); Shapley 1953 (stochastic games)
- **Optimal control theory**: Bellman 1957 (discrete/ stochastic case - Markov Decision Processes, dynamic programming); Pontryagin 1956/1962 (continuous case - maximum principle); Kalman 1960 (linear-quadratic control, filtering)
- **Artificial intelligence**: Shannon 1950 & Samuel 1959 (early trial-and-error/TD ideas in game-playing programs); Minsky 1954 (early link between learning and reinforcement); modern RL: Sutton & Barto (textbook, 1980s–1998/2018), Watkins 1989 (Q-learning), DeepMind (Atari, AlphaGo, AlphaZero)

Source: `http://incompleteideas.net/book/ebook/node12.html`

Today's applications of RL

RL can be employed in many different tasks and fields, as soon as the problem can be framed as a sequential decision-making problem with a reward function:

- **Control systems** : PID tuning, power grid optimization, traffic light control

  Ex: Konda & Tsitsiklis 2000 on average reward optimality

- **Autonomous navigation** : robot path planning, drone coordination

  Ex: Thrun et al., DARPA Grand Challenge (Stanford's Stanley); Koenig & Likhachev A*, 2002

- **Locomotion & robotics** : learning gaits, bipedal walking, manipulation

  Ex: DeepMind Locomotion, Lillicrap et al. (DDPG), Hafner et al. (PlaNet); OpenAI Dactyl

- **Language models** : fine-tuning LLMs via human feedback (RLHF)

  Ex: Christiano et al. 2017 (Deep RL from Human Preferences); Ouyang et al. (InstructGPT)

- **Game playing** : Atari, Go, Chess, Poker (AlphaGo, AlphaZero, Libratus)

  Ex: Mnih et al. 2015 (DQN); Silver et al. 2016–2017 (AlphaGo, AlphaZero)

Goals of the course

This course is a brief introduction to RL. It aims to :

1. Give you the correct vocabulary and formalism of RL
2. Give you the bases to dig into existing algorithms
3. Know how to frame (or at least how to start to frame and to solve) a RL problem and use relevant tools to solve it

Goals of the course

This course is a brief introduction to RL. It aims to :

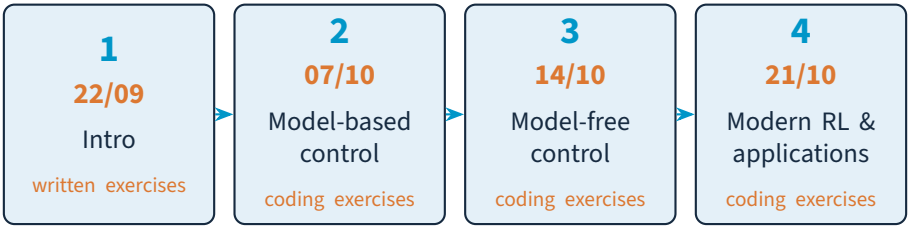
1. Give you the correct vocabulary and formalism of RL
2. Give you the bases to dig into existing algorithms
3. Know how to frame (or at least how to start to frame and to solve) a RL problem and use relevant tools to solve it

Since it is a data-driven control class, the focus is not only theoretical but also practical:

- You will implement some of the algorithms in Python.
- You will be able to use existing RL libraries to solve your own problems.

Lecture organization

The class is composed of 4 sessions of 3 hrs, each of them have 1-1.5hr of tutorials. We start from a case where environement and model is known and progressively removes assumptions on our environement:



Lecture 1: Vocabulary, MDPs, Bellman equations

Lecture 2: Dynamic programming, policy andvalue iteration

Lecture 3: Temporal difference learning, Q-learning, SARSA

Lecture 4: Function approximation, deep RL (DQN), policy gradients, applications

Learning Goals — Course 1

By the end of today's lecture, you should understand:

1. **MDPs** as the fundamental framework for sequential decision-making
2. **Value functions** (V^π, Q^π, V^*, Q^*) and what they represent
3. **Bellman equations**, the recursive structure of RL problems
4. **Optimal policies** exist and can be computed from value functions
5. **Planning vs Learning**, the two routes to solving MDPs

Why they matter? These concepts are the foundation for all RL algorithms in Lectures 2–4. They will be a 2 weeks gap between this lecture and the next one, so make sure you understand them well before October 7th!

About this course: Sutton & Barto textbook

Primary reference

Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed., MIT Press, 2018)

> *"We define a reinforcement learning method as any **effective way of solving reinforcement learning problems**, and it is now clear that these problems are closely related to **optimal control problems**, particularly stochastic optimal control problems such as those formulated as MDPs. **Accordingly, we must consider the solution methods of optimal control, such as dynamic programming, also to be reinforcement learning methods.**"*
>
> Sutton & Barto

1. A **PDF copy is available online** (link in the course materials)
2. Throughout this course, slides will reference **specific chapters and sections**
   - Example: "Sutton & Barto, Ch. 3, Sec. 3.7"

Notation

A quick reference for the symbols used throughout this course - most are defined properly later, this is just so nothing looks unfamiliar in the meantime.

$\doteq$	**"defined as"** equation
$\mathcal{S}, \mathcal{A}$	set of states, set of actions
S_t, A_t	state / action at time t
R_{t+1}	reward received when moving into S_{t+1}
G_t	return: $G_t \doteq \sum_{k \geq 0} \gamma^k R_{t+k+1}$
$\gamma \in [0, 1]$	discount factor
$\pi(a \mid s)$	policy: $\pi(a \mid s) \doteq \mathbb{P}[A_t = a \mid S_t = s]$
$v_\pi(s), q_\pi(s, a)$	state-/action-value under policy π
$*$ subscript/superscript	means **optimal**: $\pi_*, v_*(s), q_*(s, a)$ are the best achievable ones
$p(s', r \mid s, a)$	MDP dynamics: $p(s', r \mid s, a)$
$\mathbb{E}_\pi[\cdot]$	expectation under policy π

Plan

1. Introduction
2. Sequential Decision Making and RL framework
3. Markov Decision Processes (MDPs)
4. RL framework
5. Vocabulary
6. Bellman equations

Characteristics of RL

What makes RL specific :

- RL is **unsupervised**…
- …but there is a **reward function**
- The agent **observes the environment** and **selects actions** sequentially
- The goal is to learn an **optimal policy** that maximizes **cumulative reward**
- It works through multiple **trials and errors**
- Rl can learn **online** or **offline**

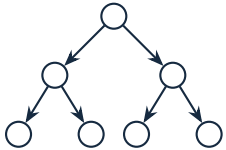
The art of decision making : Sequential decision making

Definition

Sequential decision making describes a situation where the decision maker (DM) makes successive decisions based on the observation (complete or not) of the impact of previous decisions.

Key Aspects:

- ⇒ Decisions are made in **stages over time**.
- ⇒ There is an **implicit or explicit cost** associated with each observation/action.
- ⇒ Actions can be **discrete** (a finite set of choices) or **continuous** (a real-valued range).
- ⇒ The goal is to find a **decision policy** that optimizes the decision process.



A decision tree: each stage branches into the possible actions

Important note: Sequential means that the time is discrete, but the action can be continuous. The framework says nothing about the timestep between two decisions, milliseconds, seconds, hours, days ....…

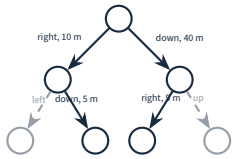
Illustrative example: cost as effort (1/2)



Navigate from *S* (start, 0 m) to *G* (goal, 5 m)

- **Problem**: reach goal *G* from *S*
- Two intermediate waypoints accessible:
  - **Right**: altitude 10 m
  - **Down**: altitude 40 m
- **Question**: which path has lower effort (cost)?
- **Cost** = sum of altitude changes along the trajectory
- Moves are **reversible**: backtracking allowed (left, up…)

Illustrative example: cost as effort (2/2)



Two direct routes (solid) vs. backtracking (dashed)

Note: Tree is actually infinite, but we only show the first two levels for clarity.

Route 1 (right $\rightarrow$ down):

- Altitude: $0 \xrightarrow{+10} 10 \xrightarrow{-5} 5$
- Effort: $|10 - 0| + |5 - 10| = 10 + 5 = \mathbf{15}$ m

Route 2 (down $\rightarrow$ right):

- Altitude: $0 \xrightarrow{+40} 40 \xrightarrow{-35} 5$
- Effort: $|40 - 0| + |5 - 40| = 40 + 35 = \mathbf{75}$ m

Conclusion: Both reach G, but Route 1 is $5\times$ cheaper!

The art of decision making: Taking optimal decisions

What does taking the best decision mean?
Being **optimal** always means optimal **with respect to a cost** (or reward) function, computed along a trajectory.

Objective functional

Given a per-step cost $c(S_t, A_t)$ (control) or reward R_{t+1} (RL), the DM optimizes the **cumulative** criterion over its **choice of actions**:

Control

$$J(a_{0:T}) = \sum_{t=0}^{T} c(S_t, A_t)$$

$$\min_{a_{0:T}} J$$

minimize cost

RL

$$R(a_{0:T}) = \sum_{t=0}^{T} R_{t+1}$$

$$\max_{a_{0:T}} R$$

maximize reward

The art of decision making: Comparing strategies and policies

Policy

A **policy** π is the decision rule followed by the DM, which is "which action to take in which state". It can be:

- **deterministic**: $\pi : \mathcal{S} \to \mathcal{A}, \quad A_t = \pi(S_t)$
- **stochastic**: $\pi(a \mid s) \doteq \mathbb{P}[A_t = a \mid S_t = s]$, verifying $\int_{\mathcal{A}(s)} \pi(a \mid s)\, da = 1$

Note: deterministic is a **special case** of stochastic, $\pi(\cdot \mid s)$ is a **Dirac mass** at $\pi(s)$, depending only on which measure is picked on $\mathcal{A}$. Definitions and proofs are thus most of the time stochastic.

The art of decision making: Comparing strategies and policies

Policy

A **policy** π is the decision rule followed by the DM, which is "which action to take in which state". It can be:

- **deterministic**: $\pi : \mathcal{S} \to \mathcal{A}, \quad A_t = \pi(S_t)$
- **stochastic**: $\pi(a \mid s) \doteq \mathbb{P}[A_t = a \mid S_t = s]$, verifying $\int_{\mathcal{A}(s)} \pi(a \mid s)\, da = 1$

Note: deterministic is a **special case** of stochastic, $\pi(\cdot \mid s)$ is a **Dirac mass** at $\pi(s)$, depending only on which measure is picked on $\mathcal{A}$. Definitions and proofs are thus most of the time stochastic.

Comparing policies

Two policies π_1, π_2 can be compared **under the same cost/reward function**:

$$J(\pi_1) \leq J(\pi_2) \implies \pi_1 \text{ at least as good as } \pi_2 \text{ with respect to the cost } J$$

An **optimal policy** π_* minimizes (or maximizes) this criterion for a given cost/reward function:

$$\pi_* \doteq \arg\min_{\pi} J(\pi) \quad \text{or} \quad \pi_* \doteq \arg\max_{\pi} R(\pi).$$

Reward

- In **optimal control**, we usually **minimize a cost** accumulated along a trajectory
- In **RL**, we instead **maximize the cumulative reward** $\sum_t R_t$
- The reward R_t is a **step function**: a single scalar handed to the agent at each time step t, not a global objective computed once at the end
- The agent never sees the full cost/reward structure in advance, it only observes R_t as it goes, **one step at a time**

Ideally, only reward we need is :

- $+1$ in case of task success
- 0 otherwise

In practice, it is often necessary to design a more informative and continuous reward function to guide the agent's learning process, this introduce a design trade-off between exploration and exploitation.

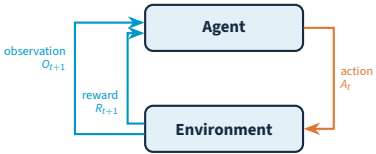
How does RL treat these sequential problems?

⇒ At each step t, the **agent**:

- Receives observation O_t
- Receives scalar reward R_t
- Executes action A_t

⇒ The **environment**:

- Receives action A_t
- Emits observation O_{t+1}
- Emits scalar reward R_{t+1}



⇒ t increments at env. step

Note: the observation O_t is **not always** the environment's (true) state S_t

In your control engineering class: in the output equation $y = Cx$, observation matrix C can be **lower rank** than x, so y alone does not determine x (Partial observation).

Plan

1. Introduction
2. Sequential Decision Making and RL framework
3. Markov Decision Processes (MDPs)
4. RL framework
5. Vocabulary
6. Bellman equations

History and state

Trajectory/History

Agent and environment interact at discrete steps $t = 0, 1, 2, \ldots$ and gives the following sequence of events:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$$

- At each step: the agent observes $S_t \in \mathcal{S}$, picks $A_t \in \mathcal{A}(S_t)$
- One step later: it receives $R_{t+1} \in \mathcal{R}$ and lands in S_{t+1}

History: everything observed so far, $H_t \doteq O_1, R_1, A_1, \ldots, A_{t-1}, O_t, R_t$
State: whatever is used to decide what happens next, $S_t \doteq f(H_t)$, the minimum set of useful information from the history

$\Rightarrow$ Goal: find the minimal state representation that is sufficient to predict what happens next, i.e. the **Markov state**

Markov state

Markov property (definition)

A state S_t is **Markov** if, by definition:
One only need S_t to predict the next state S_{t+1}, not the whole history, i.e

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]$$

Said differently, “The future is independent of the past, given the present.”

Example : On a point-mass system, position and velocity is a markov state, no need for the whole trajectory. The next state can be predicted from the current state and the action applied (acceleration), without needing to know how the system got there.

Observability

To make predictions, the agents utilize its internal state S_t^{agent} which is built from its observations O_t, being a subset of the environment state S_t^{env}.

- Full observability: the agent directly sees the environment state, $O_t = S_t^{\text{env}} = S_t^{\text{agent}}$, equivalently agent has enough information to consider the environment as a markovian system
- Partial observability: agent state $\neq$ environment state

This lecture is about **fully observable** systems.

Point of view: One actually have the choice on the state, a common trick is to include some history in the state to make it Markovian, for instance for Atari games, DeepMind used the last 4 frames as the state to make it Markovian.

Markov Decision Process (MDP)

MDP are the base of the RL formalism. They are a mathematical framework to model sequential decision making problems.

Markov Decision Process

A Markov Decision Process is a tuple $(\mathcal{S}, \mathcal{A}, p, r)$ where:

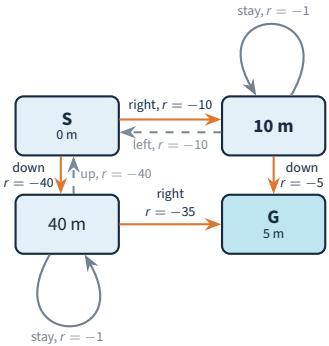
- $\mathcal{S}$: set of states
- $\mathcal{A}$: set of actions
- p : **transition probability**, $p\left(s' \mid s, a\right)$, it is also called the **dynamics function** of the environment, if this function does not change over time, the environment is said to be **stationary**
- r : **reward function**, $r\left(s, a\right) \doteq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ which is, in the most general case, stochastic
- Optionnally, $\gamma \in [0, 1]$: **discount factor** (see later)

Sutton & Barto, Ch. 3, Sec. 3.1

The 2×2 grid, framed as an MDP

S 0 m	**A** 10 m
B 40 m	**G** 5 m

Navigate from *S* (start, 0 m) to *G* (goal, 5 m)



- **Nodes** = states, **edges** = actions, labeled with the reward they give

⇒ **Reward**: $r(s, a) = -|\text{alt}(s') - \text{alt}(s)|$ - e.g. $S \to$ 10 m: $r = -|10 - 0| = -10$

- Nothing forces the agent **onward**: it can also go **back** the way it came (dashed, same $|r|$) or just **stay** at a small cost ($r = -1$, so lingering forever is never optimal)
- *G* is **terminal**: no outgoing edge, episode ends

The dynamics function of the MDP

The **joint transition–reward probability** is one way to compact the full dynamics of an MDP :

Joint transition–reward probability or dynamics function

For particular values of the random variables at time t, $s' \in \mathcal{S}$ and $r \in \mathcal{R}$, p gives the probability of those values occurring, **given** particular values of the **preceding** state and action:

$$p(s', r \mid s, a) \doteq \mathbb{P}[S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a]$$

for all $s', s \in \mathcal{S}, r \in \mathcal{R}, a \in \mathcal{A}(s)$.

Everything else (expected reward $r(s, a)$, state transition $p(s' \mid s, a)\ldots$) can be recovered from this single joint distribution p

The MDP is said to be :

- **Markov**: p only conditions on S_{t-1}, A_{t-1}, not on the full history
- **Stationary**: if p does not change over time (i.e., $p_t = p$ for all t), the environment is said to be **stationary**

Plan

1. Introduction
2. Sequential Decision Making and RL framework
3. Markov Decision Processes (MDPs)
4. RL framework
5. Vocabulary
6. Bellman equations

Policy

A policy fully defines the agent's behaviour: for every state s, $\pi(\cdot \mid s)$ is a **probability distribution** over actions,

$$\pi(a \mid s) \doteq \mathbb{P}[A_t = a \mid S_t = s], \qquad \int_{\mathcal{A}(s)} \pi(a \mid s)\, da = 1$$

- This single definition **unifies** the two cases seen so far:
  - **stochastic**: any distribution over $\mathcal{A}(s)$
  - **deterministic**: the degenerate case where all the probability mass sits on one action, $a = \pi(s)$ - a Dirac distribution $\pi(\cdot \mid s) = \delta_{\pi(s)}$

Sutton & Barto, Ch. 3, Sec. 3.5

Reward and return

Definition: reward

$R_t \in \mathcal{R} \subset \mathbb{R}$ is the scalar feedback signal received by the agent at each step t.

Definition: return

The return is the discounted sum of all future rewards from step t onward :

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $\gamma \in [0, 1]$ is the discount factor, sum might be finite if the episode terminates.

Reward and return

Definition: reward

$R_t \in \mathcal{R} \subset \mathbb{R}$ is the scalar feedback signal received by the agent at each step t.

Definition: return

The return is the discounted sum of all future rewards from step t onward :

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $\gamma \in [0, 1]$ is the discount factor, sum might be finite if the episode terminates.

What is the role of the discount factor γ?

- "Cash now is worth more than money later", this tends to prioritize medium term rewards over long term ones. $\gamma \to 0$: myopic agent, $\gamma \to 1$: far-sighted agent.
- It also ensures that the infinite sum remains finite when rewards are **bounded** and $\gamma < 1$.

Sutton & Barto. Ch. 3. Sec. 3.2–3.3

Reward hypothesis - a very old idea

Reward hypothesis (Sutton & Barto)

All goals can be described by the maximization of expected cumulative reward.

- **Galileo** (1623): the universe "is written in mathematical language" : the universe is governed by mathematical laws
- **Maupertuis** (1744), *principle of least action*: nature always acts so as to **minimize** some quantity, to some extent the first "optimal control problem"
- **Pontryagin** (1956), *maximum principle*: necessary conditions for an **optimal trajectory**, formalizing "minimize a cost along a path"

$\Rightarrow$ Three centuries of the same intuition: **behaviour = optimization of a scalar** which is echoed directly in Sutton & Barto's reward hypothesis

But...

Is such a cost/reward **always expressible**? And what happens when it is **not perfectly chosen**?

RL: finding an optimal policy in an MDP

- Once a problem is **framed as an MDP** : states, actions, dynamics and rewards, there is a well-defined notion of **optimal behaviour**

$\Rightarrow$ **RL is the search for an optimal policy** π_* for that MDP: the decision rule that maximizes cumulative reward

- The MDP is the **model of the problem**; the policy is the **solution**

Starting point: finite MDPs

This course starts with **finite** MDPs: $\mathcal{S}$, $\mathcal{A}$ and $\mathcal{R}$ are all **finite sets**.

$\Rightarrow$ $p(s', r \mid s, a)$ is then just a (finite) **table of numbers**

$\Rightarrow$ Nothing prevents the quantity from being continuous. Course will end with example of continuous state and action spaces, where p is a **density function** instead of a table

To help solving RL poblems : Value functions, definitions

The search and the evaluation of policies are based on the following functions :

State-value function

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$$

How good is it to be in state s, then follow π? v_π ranks **states** only.

Action-value function

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

How good is it to take action a in s, **then** follow π? q_π ranks **(state, action) pairs**.

- Same expected return G_t, conditioned on **less** information for v_π (just s) than for q_π (both s and the first action a)

Sutton & Barto, Ch. 3, Sec. 3.5

State-value function - example



Policy π: 50/50 at S, then straight for G

- v_π ranks **states** only - it says nothing about **which** first move (right? down?) got the agent there
- One number $v_\pi(s)$ per **cell**, under π and $\gamma = 1$:

s	S	10 m	40 m	G
$v_\pi(s)$	-45	-5	-35	0

Action-value function - example



right vs down: the two first actions compared

- $\mathcal{A}(S) = \{\text{right}, \text{down}\}$: one number $q_\pi(S, a)$ per **arrow out of** S (π, $\gamma = 1$):

a	right (to 10 m)	down (to 40 m)
$q_\pi(S, a)$	-15	-75

q_π hints at the best move

$q_\pi(S, \text{right}) > q_\pi(S, \text{down})$: **right** already looks better from S - exactly the comparison $q_*(s, a) = \max_a$ formalizes for the optimal move. This is the first brick to build the optimal policy π_* recursively.

Averaging over actions

Averaging $q_\pi(s, a)$ over $a \sim \pi(\cdot \mid s)$ gives back $v_\pi(s)$ - an **integral** over π :

$$v_\pi(s) = \int_{\mathcal{A}(s)} \pi(a \mid s)\, q_\pi(s, a)\, da, \quad v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)\, q_\pi(s, a)$$

On the 2×2 grid, at S (Start) With $\pi(\text{right} \mid S) = \pi(\text{down} \mid S) = \frac{1}{2}$:

$$v_\pi(S) = \tfrac{1}{2}\, q_\pi(S, \text{right}) + \tfrac{1}{2}\, q_\pi(S, \text{down}) = \tfrac{1}{2}(-15) + \tfrac{1}{2}(-75) = -45$$

- Exactly $v_\pi(S) = -45$ from the value table on the previous slide
- **This is Bellman's idea**: a value one step away is built from the values one step **further** away

Policy optimization

RL seek for the best policy π_* that maximizes the expected return at a any state :

Definition: optimal value functions

$$v_*(s) \doteq \max_\pi v_\pi(s), \qquad q_*(s,a) \doteq \max_\pi q_\pi(s,a)$$

The best achievable value, over **all policies**.

Policy optimization

RL seek for the best policy π_* that maximizes the expected return at a any state :

Definition: optimal value functions

$$v_*(s) \doteq \max_\pi v_\pi(s), \qquad q_*(s,a) \doteq \max_\pi q_\pi(s,a)$$

The best achievable value, over **all policies**.

Definition: optimal policy

π_* is said to be **optimal** (or to be the optimal policy) if $v_{\pi_*}(s) = v_*(s)$ for every state s. Moreover, at least one **deterministic** optimal policy always exists:

$$\pi_*(s) \doteq \arg\max_{a \in \mathcal{A}(s)} q_*(s,a) \quad \text{or} \quad \pi_*(a \mid s) = \begin{cases} 1 & \text{if } a \in \arg\max_{a'} q_*(s,a') \\ 0 & \text{otherwise} \end{cases}$$

RL is about finding π_* (or equivalently v_* or q_*) in an MDP, without knowing the dynamics p or

Sutton & Barto, ch. 3. Sec. 3.7

Theorem: Optimal Policy Existence

Main Result

For any finite MDP (finite $\mathcal{S}$, $\mathcal{A}$, and bounded rewards):

1. There **always exists** at least one deterministic optimal policy π_*
2. This policy satisfies: $\pi_*(s) = \arg\max_a q_*(s, a)$ for all $s \in \mathcal{S}$
3. The optimal value function v_* is unique, but optimal policies may not be

Corollaries:

$\Rightarrow$ We don't need to search over **stochastic** policies

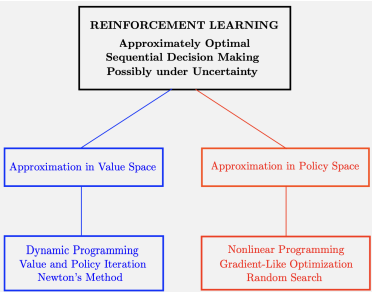
$\Rightarrow$ Once we estimate q_*, acting optimally is trivial: just pick $\arg\max_a Q^*(s, a)$

$\Rightarrow$ This justifies the focus of RL: estimating value functions, not policies directly

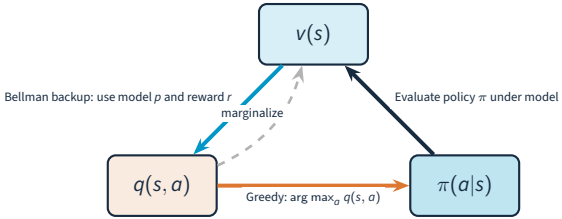
Assumptions (critical!):

- Finite state and action spaces
- Bounded immediate rewards $|r(s, a)| \leq R_{\max}$
- Stationary environment (transition dynamics don't change)

Two routes to π_*



Two families of methods: **value-space approximation** (approximate v_* or q_*, act greedily) vs **policy space approximation** (search directly over π), figure from Bertsekas, *Reinforcement Learning and Optimal Control*



The Value-Policy-Value cycle: how to transform between representations

In practice: modern algorithms often do both value and policy updates (actor-critic methods).

Plan

1. Introduction
2. Sequential Decision Making and RL framework
3. Markov Decision Processes (MDPs)
4. RL framework
5. Vocabulary
6. Bellman equations

RL taxonomy

Our class will progressively explore these different cases:

- **Model-based** (the agent has/learns p) vs **model-free** (no explicit model)
- **Value-based** (learn v or q, act greedily) vs **policy-based** (learn π directly) vs **actor-critic** (both)
- **On-policy** (evaluate/improve the policy used to generate data) vs **off-policy** (learn about a different policy than the one behaving)
- **Online** (update while interacting, at every step) vs **offline** (update afterwards: at the end of the episode, or from stored data); independent of on/off-policy
- **Prediction** (evaluate a given π) vs **control** (find π_*)

You should be able to classify any RL algorithm into one of these categories for the exam.

Vocabulary (1/2): Learning and planning

Two fundamental problems in sequential decision making:

⇒ **Planning**:

- A **model** of the environment is known, $p(s'|s,a)$
- The agent performs **computations with its model** (without any external interaction)
- The agent **improves its policy**

Example: chess engine, path planning with known map

⇒ **Reinforcement learning**:

- The environment is **initially unknown**, no access to $p(s'|s,a)$, (mostly the case in real-world problems)
- The agent **interacts** with the environment
- The agent **improves its policy**

→ We keep ideas from planning, but the agent must **learn** the model or the value function from experience.

Example: robot learning to grasp, game-playing agent learning from self-play

Vocabulary (2/2): Prediction and control

⇒ **Prediction**: evaluate the future **given a policy**

- e.g. find the value function for the uniform random policy

⇒ **Control**: optimize the future by finding the **best** policy

- find the optimal value function over all possible policies

We often do both in practive: we **predict** the value of the current policy, then **improve** it, and repeat until convergence to an optimal policy.

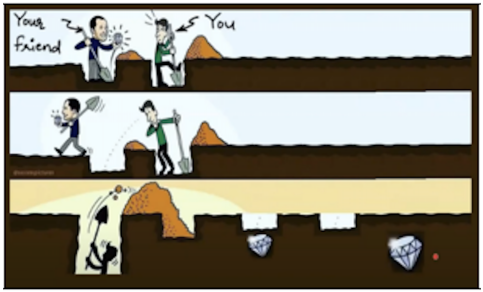
Exploration and exploitation

Reinforcement learning is trial-and-error learning, which trades off between :

⇒ **Exploration** finds more information about the environment

⇒ **Exploitation** uses known information to maximize reward

The agent must **discover** a good policy **without losing too much reward** along the way. A good agent should **explore as well as exploit**.



Exploration vs Exploitation



Plan

1. Introduction
2. Sequential Decision Making and RL framework
3. Markov Decision Processes (MDPs)
4. RL framework
5. Vocabulary
6. Bellman equations

Bellman equations

- Computing $v_\pi(s)$ straight from its definition means averaging the return over **every possible trajectory** starting from s. Their number explodes, and is **infinite** as soon as the process can run forever.
- **Bellman's idea**: don't enumerate trajectories, but exploit the **recursive structure** of the problem instead.
- Any trajectory from s is: **one step** to a trajectory from the **next state**, thus the value of "the rest" is just v_π evaluated one step later.
- This turns an **infinite-horizon** sum into a **one-step, self-referential** equation. Which is tractable even when the number of paths is not.

Bellman equations are the foundation of RL, it allows **recursive decomposition** of the value function. This allows us to compute it efficiently without enumerating all possible trajectories.

From the definition of v_π to a recursion

Definition (recall)

Value function: $v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$, Return: $G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$

- Split off the first term of the return:

$$G_t = R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} = R_{t+1} + \gamma\, G_{t+1}$$

- Take the expectation given $S_t = s$: the tail $\mathbb{E}[G_{t+1} \mid S_t = s]$ is, by definition, v_π evaluated at the (random) next state

$$\Rightarrow \qquad v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s]$$

And for q_π:

$$q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

Bellman expectation equation - v_π and q_π

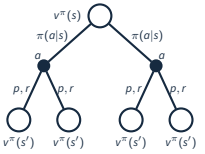
Previous equations can be rewritten in terms of the **dynamics** $p(s', r \mid s, a)$ and the policy π:

For v_π

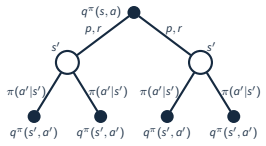
$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \Big[r + \gamma v_\pi(s') \Big]$$

For q_π

$$q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \Big[r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a') \Big]$$



π (state $\rightarrow$ action), then p



p (action $\rightarrow$ state), then π

Key insight: In finite discrete cases, these form a system of $|\mathcal{S}|$ linear equations with $|\mathcal{S}|$ unknowns, which

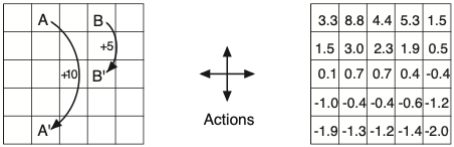
Example 3.5: Gridworld (Sutton & Barto)

Setup:

- Rectangular gridworld with 4 actions: N, S, E, W
- Deterministic transitions
- Actions that would go off-grid: stay put, reward = -1
- Other actions: reward = 0

Special states:

- **State A:** All 4 actions $\rightarrow$ reward $+10$ and teleport to A'
- **State B:** All 4 actions $\rightarrow$ reward $+5$ and teleport to B'



Gridworld layout with special states A and B

Value function under the random policy with $\gamma = 0.9$ computed by solving the system of linear equations given by previous equations.

Key insight:

- State A value < 10 (immediate reward) due to edge risks from A'
- State B value > 5 due to valued neighbors from B'

Source: Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Example 3.5

Optimal value functions v_* and q_*

The optimal value functions satisfy the **Bellman optimality equations**:

Bellman optimality equation

The value of a state under an optimal policy must equal the expected return for the **best action** from that state:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a)$$

$$v_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s', r} p\left(s', r \mid s, a\right) \left[r + \gamma v_*(s')\right]$$

Action-value form:

$$q_*(s, a) = \sum_{s', r} p\left(s', r \mid s, a\right) \left[r + \gamma \max_{a'} q_*(s', a')\right]$$

- No policy left in the equation: the max over actions *is* the optimal policy
- No closed-form solution in general

Sutton & Barto. Ch. 3. Sec. 3.6

Optimal policies

Recall :

- An **optimal policy** π_* achieves $v_{\pi_*}(s) = v_*(s)$ for **every** state s, such a policy always exists, and can always be chosen **deterministic**
- Optimal value and action-value functions are **unique**, but optimal policies may not be

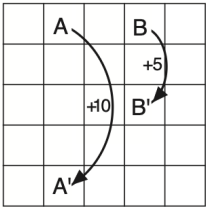
Q-learning

Once q_* is known, acting optimally is a one-line lookup:

$$\pi_*(s) = \arg\max_{a \in \mathcal{A}(s)} q_*(s, a)$$

Example 3.8: Solving the Gridworld (Sutton & Barto)

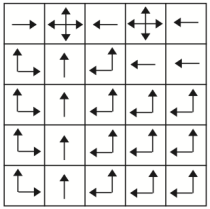
Problem: Solving the Bellman optimality equation for the gridworld from Example 3.5 give the following optimal values and policies (for $\gamma = 0.9$):



Gridworld

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

v_*



π_*

Left: Gridworld layout | Middle: Optimal values $v^*(s)$ | Right: Optimal policies $\pi^*(s)$

Key insights:

- Multiple optimal actions at some states: policy is not unique

Source: Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Example 3.8, Section 3.6

Why solving Bellman exactly is rarely the solution

Sutton & Barto's perspective: Solving the Bellman optimality equation is like exhaustive search, often impractical in real-world problems.

Three assumptions that rarely hold in practice:

1. **Accurate environment dynamics** $p(s'|s,a)$ are known
   - ✓ Gridworld, games with fixed rules
   - X Robot manipulation, autonomous vehicles, medical treatment
2. **Computational resources** are sufficient to complete the calculation
   - ✓ Small problems (4×4 gridworld)
   - X Chess has 10^{120} states, Go has 10^{170} states, real-world problems have continuous state/action spaces
3. **Markov property** holds (current state contains all relevant history)
   - ✓ Well-defined environments
   - X Partially observed problems, non-stationary environments

Consequence for next lectures: In realistic problems, we must **settle for approximate solutions**, learned from interaction rather than computed exactly, this is what RL (and this class) is about. Yet, we still use what Bellman taugght us.

Today's recap

- RL: the fact of learning to act by interacting with an environment, to maximize expected **discounted** cumulative reward
- The MDP formalism $(\mathcal{S}, \mathcal{A}, p, r)$ (plus discount γ) captures the problem; a policy π together with v_π, q_π describe its behaviour and quality
- The Bellman **expectation** equations turn "what's the value of following π" into a recursive, one-step relationship
- The Bellman **optimality** equations do the same for the best possible behaviour: v_*, q_* and the optimal policy π_*

Next lecture: solving the Bellman equations when p is known, i.e. **dynamic programming**. Dont't forget to re-read this lecture's slides before the next one.

Références I

- Sutton & Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018
- Silver, *UCL Course on Reinforcement Learning*, 2015
- Bertsekas, *Reinforcement Learning and Optimal Control*, Athena Scientific, 2019
- Abdin, *Principles and Applications of Reinforcement Learning (PARL)*, 2026
- MathWorks, *Reinforcement Learning Toolbox documentation*, MATLAB

Questions?

Your feedback on this lecture is welcome

etienne.chassaing@centralesupelec.fr

Survey form (anonymous feedback):
https://forms.gle/L7ewxBUTUaZbpcya8

Next lectures: Kahoot quiz at the start!

Reading this deck: badges

A few slides step outside the technical thread. There are meant to make you think, or to make us discuss some ideas:

- **HISTORY :** dates, people, real-world milestones (AlphaGo, Nobel prizes…)
- **ETHICS :** what can go wrong when deploying RL, and who is responsible
- **THINKING :** the ideas behind the formalism: where do they come from, and where do they stop applying?

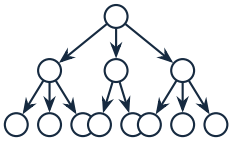
Bonus: Continuous state/action spaces

- Nothing in the Bellman equations *requires* $\mathcal{S}$ or $\mathcal{A}$ to be finite - sums simply become integrals:

$$v_\pi(s) = \int_{\mathcal{A}} \pi(a \mid s) \int_{\mathcal{S}\times\mathcal{R}} p(s', r \mid s, a)\big[r + \gamma\, v_\pi(s')\big]\, dr\, ds'\, da$$

$\Rightarrow$ **Not to be confused** with **continuous-time** MDPs (decisions at any $t \in \mathbb{R}_+$, semi-Markov formalism) - a different, much more rarely used extension; **time stays discrete** throughout this course

Bonus: why not just enumerate every game? Shannon Number



Chess: $\approx$ 35 legal moves per position, on average, $\approx$ 80 ply in a game

Source : Wikipedia, Shannon number

- Chess is, formally, just a (huge) **finite** MDP: states = board positions, actions = legal moves

$\Rightarrow$ The full decision tree has roughly $35^{80} \approx 10^{123}$ leaves - the **Shannon number**

- Exhaustively planning through every branch is **intractable**: more leaves than atoms in the observable universe

Takeaway

Enumerating the tree is a dead end for anything but toy problems. RL's whole bet: **never** build the full tree - learn value estimates and explore **cleverly**, visiting only a tiny, well-chosen fraction of it.

Notes on AI and LLM

About this course: I do use LLMs/AI to create this course, with the following uses:

- Format LaTeX slides and exercises
- Find relevant sources
- Format equations and notations

However, plan and ideas are human-based, and every content has been carefully reviewed. If you find some errors, or inconsistencies, please report them to me.

What about your own LLM usage?

Of course you also have the right to use LLMs, but don't forget you're in a learning step. Learning process goes through effort and the newer the subject, the more one needs empty-sheet recall.

* See: Desirable difficulty
* Chi & Wylie (2014), ICAP hypothesis